Editing the editor: writing an extensible language extension for Agda (Research proposal)

Job Vonk
Utrecht University
j.vonk@uu.nl

1 Introduction

Dependently-typed programming languages provide a means to prove the absence of bugs. This gives rise to code bases where the functionality is completely verified, that is, the use of the library or a program cannot go wrong according to their defined specification. Even though complete verification sounds ideal for all code bases, there is one major flaw: using formal methods, like dependently-typed languages and theorem provers, to verify code can be intensive. It takes mathematically skilled people, and quite some time (compared to informal quality assurance methods like testing) to fully verify a system [1].

Juhošová et al. [2] have investigated what obstacles students experience when learning Agda as an interactive theorem prover. Their advice is that improve robustness and accessibility, where one point they mention for both improvement areas is polishing and improving the existing tooling. Backx [3] has also investigated how novice dependently-typed language programmers write code, but also includes experienced programmers in his study. He saw that users experience a lack of tooling support for refactorings, and also recommends authors of dependently-typed interactive theorem provers to improve the editor tooling with, for example, a search feature for lemmas, or features that can also be found in non-dependently typed languages.

Focusing on Agda in particular, we see little innovation in editor features: all of the extensions for all currently available editors support roughly the same feature set. One reason for this might be that extending this feature set is quite difficult to do: the features are implemented within the Agda compiler, which is a large and complex Haskell code base.

Another reason for the little innovation might be the fact that none of the extensions are written in Agda itself, rather they are implemented in various languages like ReScript, Haskell or Emacs Lisp to name a few. This raises the barrier of entry for people familiar with Agda wanting to contribute a new extension feature.

These two observations motivate the following research questions:

1. **Is it possible to make an extensible Agda extension by writing it in Agda itself?**
   We will show this by writing an Agda language extension for Visual Studio Code in Agda that is extensible by adding extra new features onto it, which can also be written in Agda. Evaluating the extensibility of the extension is quite difficult, since there exists little comparison material. One rudimentary way to evaluate the extensibility of the extension is by measuring the number of extra lines of code required to add new features.

2. **Is Agda a suitable language to write practical programs with?**
   Current big agda projects are libraries like the standard library, unimath[1] and cubical[2], which are impressively large and highlight Agda's strength as a theorem prover. However, at the Agda wiki's home page, the language is introduced as a dependently-typed *programming language* before being introduced as a proof assistant[3]. Strangely, there does not seem to be a big practical programming

[1] https://unimath.github.io/agda-unimath/
[2] https://github.com/agda/cubical
[3] https://agda.readthedocs.io/en/latest/getting-started/what-is-agda.html

project showing Agda's viability as a programming language. Writing a language extension in Agda uncovers whether or not Agda is a viable *programming language* as opposed to a theorem prover. After the development, we can reflect on issues in the language or its tooling, and report a list of improvements to make Agda more suitable for practical applications.

2 Background

2.1 Existing Agda extensions

The Agda ecosystem has a number of extensions for different programming editors. All of these extensions behave very similarly and feature a similar set of tools to Agda developers:

- There are numerous commands the developer can issue using key binds to navigate goals, request a goal's type and context or refine parts of the goal.
- Additionally, the extensions are responsible for providing semantic syntax highlighting. That means that tokens are highlighted based on what they represent, for example, identifiers referring to functions are highlighted in blue, whereas identifiers for constructors are green.
- Lastly, the extensions provide an input method for writing unicode characters in the Agda source files. The developer can type `"\"` and a identifier for the character (which is usually the latex code), to write characters that cannot be written using regular keyboards.

There are various implementations of Agda extensions for different editors. These implementations can use the endpoints provided by the Agda compiler for external programs to call Agda internals. The compiler has flags which open a Read-Eval-Print-Loop (REPL) environment that extensions can use to issue commands and receive responses.

- emacs-mode is the first Agda extension, which was developed alongside the Agda compiler. The extension is divided into two parts:
  1. The user event listeners, these are functions triggered by command hooks within Emacs. These hooks send different commands to the Agda compiler, depending on the type of action. The extension opens a handle to an Adga process with the `--interaction` flag, that the extension communicates with.
  2. Agda responds with Emacs lisp code that triggers response functions. These response functions are parameterised by data that is provided by the Agda compiler, which encodes the response data in Emacs lisp data types.

  Interestingly, neither emacs-mode nor the Agda compiler holds any state about the editor. Emacs does internally store the ranges and positions of the tokens provided by Agda, and also does the diffing when edits are made.
- Cornelis is an extension for vim written in Haskell. As opposed as emacs-mode, it uses the the `--interaction-json` flag, which has the same API as the `--interaction` flag, but responds with a JSON format instead of Emacs lisp code.

  Cornelis also has a different way of handling responses and managing the communication to the Agda compiler. It creates a process handle for each buffer, such that responses can be processed asynchronously, and disregard the editor's state.
- Agda-mode-vscode by banacorn is the original extension for Agda in Visual Studio Code (VS Code). It is written in ReScript, an OCaml-like language that compiles to JavaScript. It uses the `--interaction` compiler flag, since it predates the `--interaction-json` flag.

  This extension tries to emulate the behaviour of the emacs-mode, but VS Code has less builtin machinery for tokenisation especially when the buffer has been edited. This means the extension listens for edits to the buffer, and recalculates the positions of the known tokens to keep the

highlighting working for unchecked states. Additionally, the extension also features a system to facilitate the installation of Agda.

The VS Code extension also has integration with the Agda Language Server (ALS). It is a language server for Agda created by the same author as the VS Code extension. The implementation currently only contains a hover functionality that shows the type of a term once the file has successfully been type-checked.

- Recently, a new VS Code extension for Agda has been released by Benjamin Driscoll[4]. This extension is written in Typescript – the native language of the VS Code APIs. The functionality of this extension is mostly the same as the banacorn's VS Code extension, apart from some bug fixes and user interface changes.

2.2 Visual Studio Code

Visual Studio Code is an open-source code editor developed by Microsoft. According to last year's StackOverflow Developer survey, 79.6% of the respondents used Visual Studio Code regularly in 2025 [4]. While the editor ships with support for Javascript/Typescript support out of the box, it has a robust extension system and a marketplace with extensions for many other programming languages. This includes extensions for mainstream languages like C, C#, Rust, etc. However, Visual Studio Code has also been target for dependently-typed languages. In particular Lean4 has builtin support for LSP, and has a very sophisticated extension.

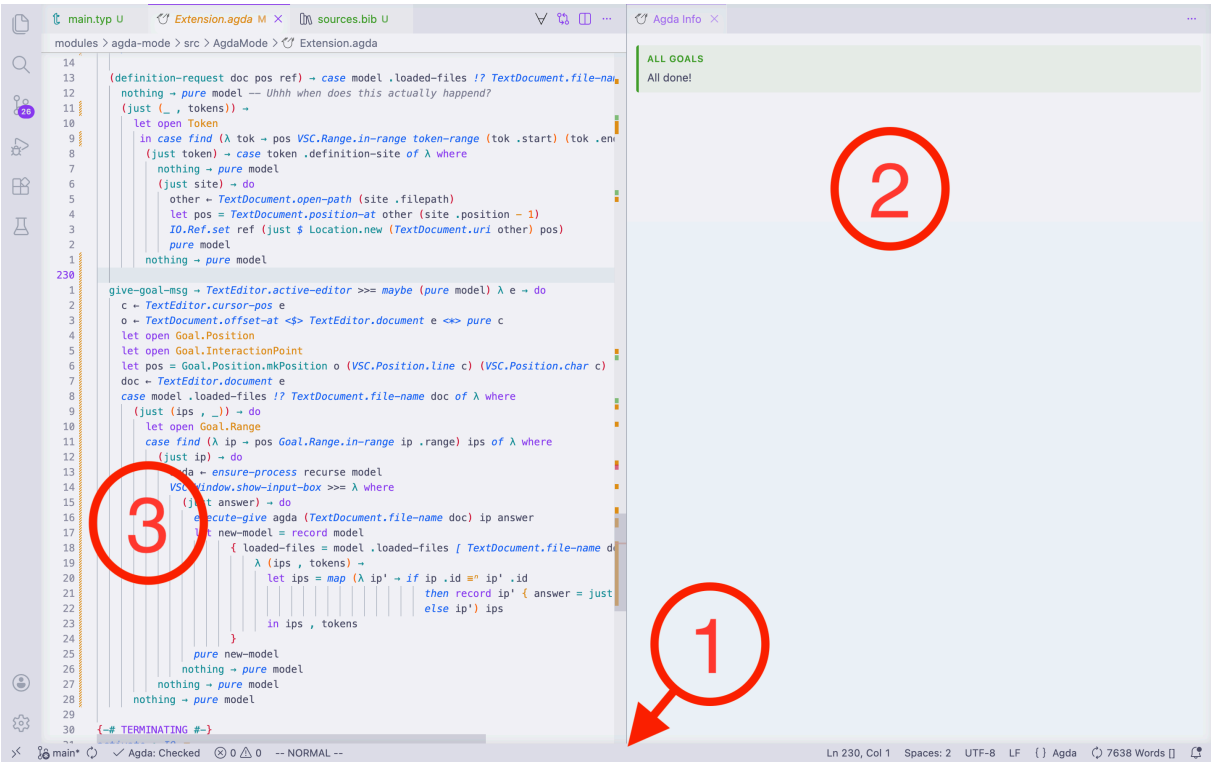

Figure 1: Screenshot of Visual Studio Code, the numbers correspond to the enumeration in the paragraph below.

The editor has a number of configurable elements that can be controlled via an API:

1. There is a status bar at the bottom of the screen that displays small amounts of information – typically single words or icons.
2. The API allows extension developers to create panels, these are implemented as an HTML `iframe`. This means that the panel can display arbitrary HTML, CSS and Javascript. Additionally, the

[4] https://github.com/willtunnels/agda2-vscode

extension can send and receive messages to and from the panel using the `window`'s `postMessage` method and the `window`'s `message` event.

3. The editor itself can also be interacted with. Extension developers can request parts of the buffer based on positions within the document. Or the buffer can be programmatically edited, or decorated using syntax highlighting, or CSS classes (for underlining text, for example).

2.3 Language Server Protocol

The Language Server Protocol (LSP) is a protocol developed by Microsoft that aims to standardise the communication between language tooling and integrated development environments (IDEs) [5]. The purpose of the protocol is to solve the $M \times N$ problem for language extensions. The essence of this problem is that language tooling developers need to implement a separate extension for each editor they want to support. As shown in Figure 2, given N languages and M editors, this results in $M \times N$ extensions in total. LSP simplifies this by letting language tooling developers only implement the language side of LSP, and editor developers implement the editor side of LSP. This reduces the total number of extension down to just $M + N$, which is significantly less burden on tooling developers.

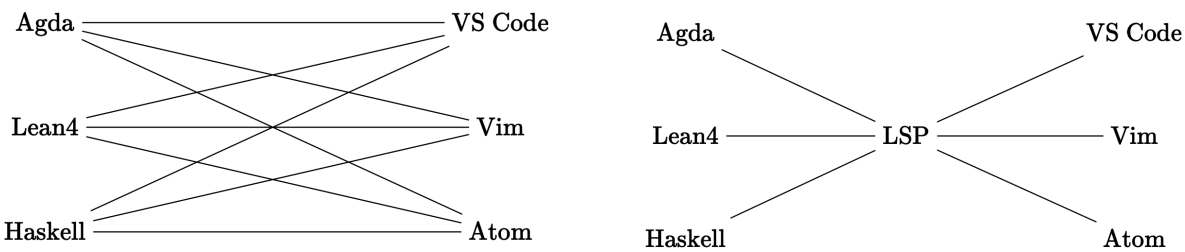

Figure 2: Visualisation of how LSP solves the $M \times N$ problem for language editors: each language tooling author needs to support just LSP instead of every single editor.

In more technical detail, the editor sends notifications to the language tool about actions the user has done, e.g. opening or closing a document, changing the source code, or executing a code action. The language tool can respond to those notifications as well as send notifications to the IDE about warnings, errors or definition sites.

The LSP protocol has been proven in practice by many main stream languages, among which TypeScript and C#. Even dependently-typed languages have implementations for the LSP protocol:

- Idris has a Visual Studio Code extension that supports the LSP protocol [6]. There is a package `idris2-lsp` that depends on the Idris compiler. Since both the LSP server and the compiler are implemented in Idris itself, the LSP can depend on the compiler directly. And therefore, the LSP can immediately invoke the type checking commands. The Idris LSP has some limitations, however. One of those is that hover actions only work if the entire file type checks.
- Lean4 has an implementation of the LSP protocol with Lean-specific extensions. The implementation is written in Lean itself as part of the compiler [7]. The Visual Studio Code extension, for example, can rely on the core LSP part for providing autocompletions and code actions. However, since the extensions are non-standard, they require implementations for every IDE separately. For example, the `vscode-lean4` extension has workspaces for setting up the LSP, as well as the infoview window, which displays context information, results of `#evals` and goal types.
- Rocq has two extensions for Visual Studio Code, each with their own implementation of the LSP protocol: `vsrocq` with the `vsrocq-language-server` [8], and `rocq-lsp` [9]. The `rocq-lsp` GitHub repository describes that, in addition to the standard protocol, there are interactions specific to Rocq for, for example, displaying the proofs goals in tactic mode and file checking progress updates. Similar to Lean, both extensions also feature a infoview window where the proof goals, context and `Print` statements are displayed.

The extensions these languages have made are all quite similar of nature, and Rask et al. [10] have therefore developed the Specification Language Server Protocol (SLSP) that aims to standardise extensions to the LSP protocol regarding theorem proving, providing goal info and translating to other languages. They have implemented a server for SLSP for the Vienna Development Method, which is a specification language that also features theorem proving.

3 Preliminary results

Currently, we have a working prototype implementation of a Visual Studio Code extension written in Agda – which compiles to Javascript – that supports a subset of the features other Agda extensions also have. Section 3.1 discusses what features the current version of the extension supports, and Section 3.2 discusses the architecture of the extension in further detail, whereas Sections 3.3, 3.4, 3.5 and 3.6 go into major problems that appeared during the prototype's development, and how they were solved.

3.1 Current capabilities

Figure 3: The current version of the extension has a display buffer that can show progress of type-checking as well as goals and errors (left). If the file type-checks, then the file is syntax-highlighted (right).

The current iteration of the extension only exposes the "load file" command for users to invoke. Even though there is support for one command, invoking it, gives the extension all the information it needs to provide many different features which are also supported in the current version of the extension:

- *Display buffer*: when the load file command is invoked the compiler shares information about its progress type-checking the current file and its dependencies. This information is displayed in the display buffer. After type-checking completed, the buffer can show either information about goals – if the type-checking was successful – or it can show information about type errors in the current file or its dependencies.
- *Syntax highlighting*: Once the current file is successfully type-checked, the compiler also shares highlighting information to the extension. This is converted to the format Visual Studio Code expects, and the new highlighting is applied to the editor buffer.
- *Jump to error*: if the type-checking was not successful, then the compiler will point to the type-error it got stuck on. The extension receives information about where this error is located and moves the user's cursor to that file at the correct position.
- *Go to definition*: The extension answers go-to-definition requests from the editor, where definition site information is provided by the `HighlightingInfo` message. Visual Studio Code provides a position, and the extension has to query the tokens for the currently active file to find one which has the position in its range.

- *Input mode*: As mentioned in Section 2.1, all extensions have support for the unicode input mode to type unicode characters in Agda source code. This extension also has support for the input mode, in a similar fashion as the emacs-mode, where the status bar shows which characters can be typed next, as well as show which characters can currently be selected using the arrow keys.

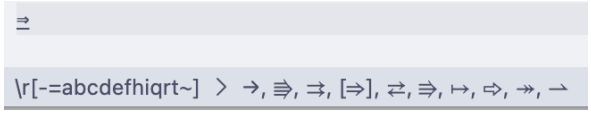

Figure 4: The status bar during input mode shows which characters have been typed, which characters can be typed next, and which variants of the current character there are.

3.2 Extension architecture

The extension relies on the Agda compiler's interaction mode. This mode has two response formats: Emacs Lisp – which was initially made for the Emacs-mode – and JSON designed for non Emacs-Lisp extensions. Given that Javascript has a builtin function for parsing JSON, which the extension can make great use of during the prototyping phase, the extension uses the JSON-based interaction mode.

Below we will describe how the "load file" command is handled as an example of how interactions with the interaction mode go. Given this is the only command the extension currently supports, it showcases well how all of the components in the architecture work.

The user has to initiate interactions with the extension. That is, the extension is idle as long as the user does not issue any commands. Once the user uses the `C-c C-l` keybind, the extension receives a message and immediately issues the `Cmd_load` command to the Agda compiler.

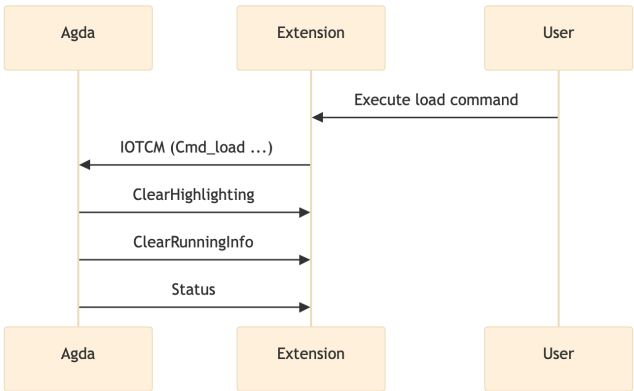

Figure 5: Initial requests and responses when the user issues the "load file" command.

The Agda compiler initially responds with messages for clearing the token cache the extension internally keeps track of, empty the display buffer, and clear the status bar items.

After the initial messages, the compiler sends updates while it is type-checking the requested module and its dependencies. The `RunningInfo` message appends new lines to the display buffer. The `DisplayInfo` message replaces the display buffer's content completely.

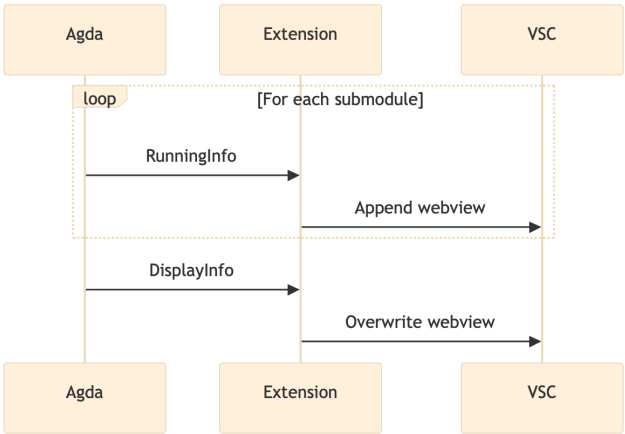

Figure 6: The responses from the Agda compiler updating the display buffer.

Once the compiler finishes type-checking, it responds either with error messages, or with highlighting info. The error message redirects the user's cursor to the line where the error occurs, and set the error in the display buffer. The highlighting message contains a list of tokens and their semantic token type, as well as

information of definition sites which can be used in the go-to-definition feature.

The extension notifies Visual Studio Code that there is new highlighting information, and Visual Studio Code sends a token request to which the extension responds with the newly retrieved information. When the user edits the buffer, Visual Studio Code requests new token information, even though the compiler has not provided any new tokens.

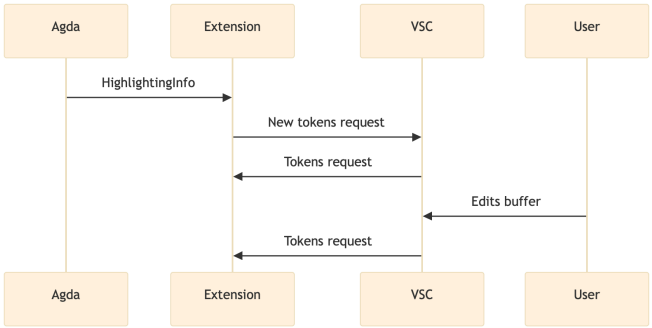

Figure 7: When the file type-checks, the compiler responds with highlighting information. When the user edits the buffer, Visual Studio Code requests new tokens.

There is one component of the extension that operates completely separately from the compiler: the input mode. The extension listens for Visual Studio Code's `type` event, which is fired on every character that should be displayed in the editor. If the character is `\`, then input mode is enabled, and any subsequent characters are captured and move the focus in a trie zipper that contains all of the replacements. If the code matches a value in the trie, then it is replaced in the editor, until the user types something that is not matched by the trie, which exits the input mode.

3.3 Importing Javascript libraries

The extension uses some external libraries that need to be imported in the Javascript output, in order to function. Usually in Javascript, the `import` statement is used to import libraries. However, the latest release of the Agda compiler does not provide a way to import Javascript libraries. Instead the extension relies on a separate Javascript file to set the dependencies in a global variable available across all modules, and then call the extension's main function, as shown in Listing 1.

```javascript
import Main from "./jAgda.AgdaMode.Extension.mjs";
import * as vscode from "vscode";

export function activate() {
  Object.assignProperty(globalThis, "AgdaModeImports", {
    value: { vscode, /* and some other libraries */ }
  });
  Main(); // Call the main function from the extension manually
}
```

Listing 1: A Javascript file importing libraries and making them globally available before running the Agda code.

The Agda compiler does have a pragma for inserting arbitrary code at the top of the output files, however, it is currently only implemented for the GHC backend. Implementing the `FOREIGN JS` pragma requires very little changes to the Agda compiler, and we created a fork of the Agda compiler that includes support for the pragma[5]. This allows the modules to import their Javascript dependencies without requiring a separate file to bind them to the global scope. An example use of the pragma is shown in Listing 2.

[5] https://github.com/agda/agda/issues/8429

```
{-# FOREIGN JS import * as vscode from "vscode" #-}
postulate alert : String → IO ⊤
{-# COMPILE JS alert = text => async () => {
  vscode.window.showInformationMessage(text);
  return a => a["tt"]();
} #-}
```

Listing 2: Using the `FOREIGN JS` and `COMPILE JS` pragmas to define an Agda function for showing a notification in Visual Studio Code.

3.4 Handling asynchronicity

By default Agda expects IO operations to use continuation-passing style, though the builtin module does not provide the monadic operations: `pure` and `_>>=_`. When compiling, the Agda compiler looks for a `main` function, and if it exists, it inserts `main(() => {})` at the end of the file. The standard library author, therefore, has a choice how to implement these functions as long as calling the function starts the `IO` operation.

Initially, the extension made use of continuation-passing style for `IO` (provided by the Iepje library[6]). However, Visual Studio Code exposes some functions that return promises: a type that modern Javascript uses to handle asynchronous computations. A promise represents a computation that has already started and will return a value in the future, an example of which can be found in Listing 3.

```javascript
const delay = (time, f) => new Promise((resolve, reject) =>
  setTimeout(
    () => resolve(f()),
    time
  ));
delay(1000, () => 10).then(n => console.log(n)); // Prints 10 after 1 second
```

Listing 3: Example function that uses promises that delay a computation and waits for the computation to finish to print the result.

To be able to use these functions from the Visual Studio Code API, the extension requires a different definition for `IO`, where instead of compiling to continuation passing style, we use lazy promises. Using just promises for the `IO` monad – with the `new` constructor as `pure`, and `.then` for `_>>=_` – means that any term of type `IO A` has already started executing. Since the extension makes abundant use of "lazy" `IO`, where storing the `IO` action does not mean it has already started executing, we have decided to define `IO` as functions of Typescript type `<T>() => Promise<T>`.

However, in order to avoid stack overflows when, for example, traversing large lists with the `IO` monad, we need to make use of the syntax sugar that Javascript has for promises: `async/await`. As seen in Listing 4, an implementation with `async/await` is a little more verbose, but reduces the stack depth of nested `IO` operations compared to using `.then` directly – which adds a stack frame every time it is called.

[6] https://github.com/lawcho/iepje

```
ma => f => async () => {
  const a = await ma();
  return await f(a)();
}
```

```
ma => f => () =>
  ma().then(a => f(a)())
```

Listing 4: Juxtaposition of an implementation of _>>=_ with stack depth linear to the nesting depth of the _>>=_ operations (left), and an implementation with stack depth linear to the number of evaluated _>>=_ calls (right).

3.5 Passing messages to the webview safely

As mentioned in Section 2.2, Visual Studio Code has the ability to open a panel in which any webpage can be displayed. Internally, Visual Studio Code has a wrapper API around an `iframe` HTML element. This API includes the ability to send and receive messages between the webview and the extension. There is one caveat mentioned in the documentation of Visual Studio Code: the objects sent to and from the webview must be JSON-serialisable[7]. This means that only objects that can be serialised to and deserialised from JSON can be passed. Even though this limitation is not statically enforced by Visual Studio Code's Typescript API, it can be enforced in the Agda API as shown in Listing 5.

```
-- Class of JSON-serialisable types
record Cloneable (A : Set)
  : Set where field
    encode : A → JSON
    decode : JSON → Maybe A
    encode-decode-dual : ∀ a →
      decode (encode a) ≡ just a
```

```
data JSON : Set where
  j-null : JSON
  j-string : string → JSON
  j-bool : boolean → JSON
  j-number : number → JSON
  j-array : List JSON → JSON
  j-object : StringMap.t JSON → JSON
```

Listing 5: Encoding of the JSON-serialisable property in Agda: a type is JSON-serialisable when it can be encoded to and decoded from JSON, and the data is unchanged after encoding and decoding it.

The `JSON` data type has a custom foreign function interface, where each constructor of the data type is represented by its JSON primitive in the Javascript runtime. This means that any term of type `JSON` can be passed as a message to a webview, and as such anything that can be serialised (via `encode`) and deserialised (via `decode`) correctly (formulated as `encode-decode-dual`) to and from `JSON` can also be passed using these functions.

What remains is to use this type class in the `Panel` foreign function interface, as shown in Listing 6. The `Panel` type becomes polymorphic over the message type in Agda, and when creating an instance (via `create`), it is required that the message type is `Cloneable`. It is worth pointing out that in the Visual Studio Code documentation the `Webview` type (the type `Panel` is modelled after) is not polymorphic and any object is allowed to be passed, resulting in unexpected data at the receiving end[8].

```
module Panel where
  postulate t : Set → Set
  create : { A : Set } {{ Cloneable A }} → String → String
         → ShowOptions.t → WebviewOptions.t → IO (t A)
  send-message : { A : Set } {{ Cloneable A }} → t A → A → IO ⊤
```

Listing 6: Using polymorphism and the `Cloneable` type class to ensure messages are JSON-serialisable – a constraint that is not enforced statically by the Typescript API.

[7] https://code.visualstudio.com/api/extension-guides/webview#passing-messages-from-an-extension-to-a-webview
[8] https://code.visualstudio.com/api/references/vscode-api#2438

3.6 Mitigating stack overflows

The Agda compiler does not support tail-call optimisation. Since recursion is the only method of traversing a data structure, this means that the stack size determines how deep we can traverse a data structure in one pass. For the implementation of the extension this poses a serious problem, since the responses from Agda containing the highlighting tokens grow linearly with the size of the source file that is being checked. For relatively small source files, we already run into the stack limit.

To mitigate this, we need to use the JavaScript ffi to define "stack-safe" functions that can traverse large data structures without increasing the stack depth per element. Agda allows us to provide a definition that is used by the elaborator, and another separate one that is used in the compiled output. This means we retain the ability to reason over the functions, as well as removing the worries about the stack depth.

```
reduce-right : B → (B → A → B) → List A → B
reduce-right b f [] = b
reduce-right b f (x ∷ xs) = f (foldr b f xs) x

{-# COMPILE JS reduce-right = a => A => b => B => b => f => xs =>
  xs.reduceRight((ac, e) => f(ac)(e), b) #-}
```

Listing 7: The implementation of the `reduce-right` function both for Agda's elaborator and for the compiled output. The Javascript definition offers stack-safety.

While this method does resolve stack-safety issues, we do have another problem. Agda takes the `COMPILE JS` pragma and inserts it unchecked in the compiled output. Therefore, there is no guarantee that the definition in the pragma adheres to the type declaration, and that the pragma definition has the same semantics as the Agda definition.

A pragmatic solution to this "consistency checking" problem is to create a separate program that does the following:

1. For each test case, the program requests a fully qualified function name, and test inputs for each relevant parameter. The program will compile the library and find the correct Javascript definition in the compiled output.

   ```
   run_consistency_tests
     name: "Data.List.map"
     inputs: cartesian_product test_functions, test_lists
     erased_params: 4
     meta_information: "{A = Nat} {B = Nat}"
   ```

2. The given inputs are applied to the function, taking the erased parameters into account, and then they are serialised together with the output for the next step.
3. The outputs from the compiled JavaScript code are inserted into a temporary Agda module that prompts the elaborator whether the the same function call gives the same output. In particular, the tester creates Agda source files with the structure described in Listing 9.

```
module Test where
-- Import the module containing the function we want to check.
open import Data.List
open import Agda.Builtin.Equality
-- For each test case, there is an assertion that the Agda definition
-- normalises to the same output as the JavaScript definition did.
_ : map (λ x → x + 1) (1 ∷ 2 ∷ 3 ∷ []) ≡ (2 ∷ 3 ∷ 4 ∷ [])
_ = refl
```

Listing 9: Agda file for checking consistency once the output of the JS definitions is collected.

If this file type checks, it means that for each test case, the Agda definition is return the same results as the Javascript definition. Otherwise there might be issues in i.a. the number of parameters, different semantics, or type errors in the Javascript definition.

With well-chosen test cases, there is a reasonable degree of consistency between the Agda and JS definitions. By transitivity, the JS compiled output has a similar degree of correctness if the Agda definition is verified. With this method, we have a way to interact with the Javascript FFI in such a way that we have some degree of quality assurance.

There are some limitations to this tool, especially when testing functions operating on more complex data types, or function using type classes. Data types and type classes are compiled to more complex objects, and may make use of external functions whose structure and generated names are quite hard to predict.

4 Planning and remaining work

There are a number of ways we can continue on the preliminary results. The sections below each describe a way to continue the project. They have been sorted in order of when we want to complete them:

- We aim to finish the features in Sections 4.1 and 4.2 within this thesis. In particular, Section 4.1 should be done around the end of March. This gives plenty of time to implement Section 4.2, which require changes in the Agda compiler, which might take time to setup a development environment for, and to understand the internals of.
- If one of these features appears to be impossible or unreasonably difficult to implement, we can look into Section 4.3: we expect to not have too much trouble implementing a new editor and separating the editor-specific implementation from the "core" logic.
- Sections 4.4 and 4.5 are out of scope for this thesis: they are either too tangential on the preliminary results, or have been researched enough that we do not expect to give meaningful novel results in the remaining time for this thesis.

4.1 Making the extension feature-complete

The current prototype is a showcase for a wide variety of systems that comprise Agda extensions. What remains to be done for the extension to be on par with the extensions for other editors is the following:

- *Commands relating to goals and interactions with the elaborator*: with the current infrastructure, these should not be hard to implement. The only new system that these commands require is an input field for the user to enter Agda code to fill a goal with, or to determine the type for, etc.
- *Handling edits in buffers*: As described in Section 3.2, Agda only provides tokens if the file type-checks and the load command is issued. This means that when the user edits the buffer, we cannot request tokens from Agda for the edited buffer, which likely contains type errors. Additionally, since

type checking in Agda is quite computationally expensive, we would like to sparingly send load commands.

What other extensions have done to provide as much highlighting as possible is to keep track of the edits in the buffer and shift the positions of the tokens in the cache, and possibly invalidate or remove tokens when there are edits within them. Doing this allows the highlighting and the go-to-definition service to work for most of the file, until the file is loaded again.

Concretely, we can take inspiration from the `diff-loc` library[9]: we can keep track of edits (insertions, replacements and deletions) in a sequence ordered by line and column ascendingly. If we want to translate a source location in the unedited buffer to one in the edited buffer, then we apply each edit from the start of the sequence until the edits involve lines after the requested source lines.

- *Handling remaining display messages*: the currently handled subset of the display messages are the most common, and occur frequently during the informal testing while developing the extension. There are some rarer messages that are sent by Agda when certain compiler flags are set
- *User settings*: most extensions allow the user to configure the path to the Agda binary, and which flags should be passed in addition to the flags the extension passes.

4.2 New features and extensible IDE

Given a feature-complete extension, a natural question is are there any new features that can be implemented in this extension? The problem is that the current architecture for Agda extensions is rather limiting. With the current setup, we need to either implement the new features directly in the Agda compiler, or make a separate tool in Haskell that depends on the Agda compiler. Both approaches require us to learn the internals of the Agda compiler, which can pose a high barrier of entry for Agda programmers who want to make their own editor tooling.

4.2.1 Building an API for an extensible language extension

Instead of modifying the compiler to add editor features, Korkut and Christiansen [11] have implemented an extensible editor for Idris1 where editor commands can be defined within Idris1 itself using the metaprogramming library. The procedure they used to implement the editor extensions can be translated quite well to Agda. They describe the following steps:

1. Define an `Editorable` type class that captures (de)serialisation of messages from the editor. The type class should have a function for converting `JSON` to a term, and a function for converting a term to `JSON`.

```
record Editorable (A : Set a) : Set a where field
  from-editor : JSON → TC A
  to-editor   : A → TC JSON
```

Listing 10: Possible definition of the `Editorable` type class for (de)serialising reflection types.

2. We define `Editorable` instances for data types in Agda's reflection library. There is one caveat with definiting these instances, as remarked by Korkut and Christiansen: users of an IDE write terms in Agda's surface language, whereas the reflection library operates on a lower-level language. They mention that an easy to way to bridge the gap between these languages is to define the instances for the reflection library as primitives in the compiler. The primitives can have the signature as described in Listing 11.

[9] https://hackage.haskell.org/package/diff-loc

```
data HasPrim : Set → Set where
  has-Term : HasPrim Term
  has-Definition : HasPrim Definition
  {- ... -}

primitive
  primFromEditor : HasPrim A → JSON → TC A
  primToEditor   : HasPrim A → A → TC JSON

instance
  Term-Editorable : Editorable Term
  Term-Editorable = record
    { from-editor = primFromEditor has-Term
    ; to-editor = primToEditor has-Term }
```

Listing 11: Signatures for the `from-editor` and `to-editor` functions for the builtin reflection data types.

The implementation of the primitives need to be completed in the compiler. Ultimately, the IDE user can call an edit-time function with some terms as arguments. Figure 8 describes on a high-level the steps that need to be taken to run the edit-time function.

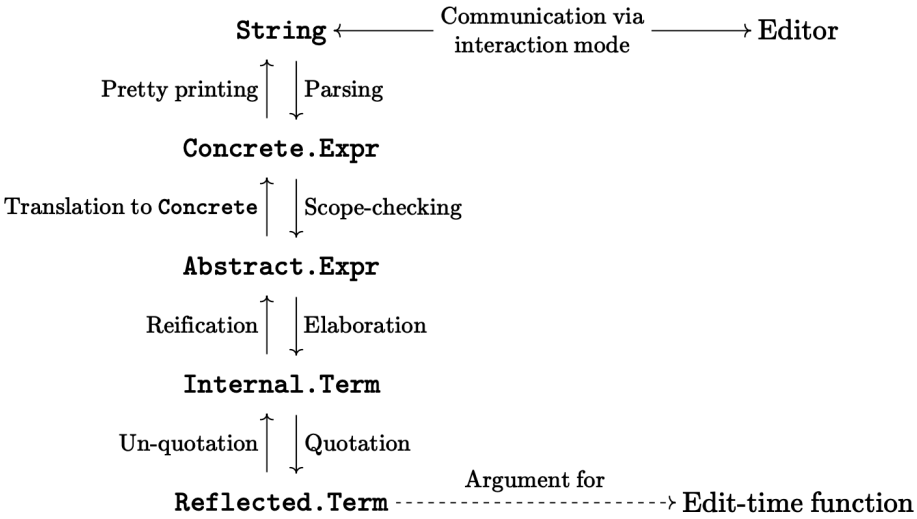

Figure 8: Visualisation of the pipeline to pass terms from an editor extension to the compiler to be used in an edit-time function.

In summary, the IDE user writes surface level terms, we can send those as strings to the compiler and let it parse it to `Concrete.Exprs`. The compiler can scope-check and type-check these `Concrete.Exprs` to `Internal.Terms`, which can be `quoted` and passed as arguments to edit-time function. The result goes through the inverse pipeline: the `Reflected.Terms` are unquoted, reified, pretty printed and sent back to the extension, which can insert the resulting term into the buffer.

3. Add a language pragma or modifier keyword to mark a function as being an edit-time function. During type-checking the compiler needs to check whether a edit-time function is a valid, which is when:
   - each of the parameters have an instance for `Editorable`, in order to decode the parameters from the editor request;

- the return type is in the `TC` monad, and its result is also `Editorable`, in order to communicate back the result to the editor.

4. Extend the interaction mode such that on a successful file load, all of the edit-time functions are listed. Within Visual Studio Code, these edit-time functions can be displayed as code actions, which are listed under the "refactorings" menu when right-clicking a piece of code. The benefit is that Visual Studio Code has additional features for refactorings that need not be implemented in the extension itself. For example, it allows programmers to bind specific code actions to a keybind using a workspace or global configuration file, or it allows programmers to preview refactorings before actually applying them to the buffer.

   When a code action is invoked, a command must be sent to the interaction mode. This means, the interaction mode should also be extended with a new command that runs a edit-time function with arbitrary arguments supplied.

4.2.2 Integration of edit-time functions with the extension

In addition to implementing edit-time functions in Agda with support from our extension à la Korkut and Christiansen, we can make use of the fact that our extension itself is also implemented in Agda. The edit-time function in Idris1 required some "glue" code written in Emacs lisp – the language the editor extension is written in. Our extension supports Agda end-to-end: an Agda metaprogram is used to define the edit action, and the way to call the metaprogram and handle its response in the editor is also written in Agda. The benefit is that the edit action and handler can be co-located, and require understanding of only one language, which both help maintainability of extension features and a lower barrier of entry for Agda programmers to write their own features.

This benefit can be fully exploited when the extension also provides an easy way for feature authors to integrate their feature into the extension, ideally without having to touch parts of the extension's source code. Therefore, after having completed the implementation of edit-time functions, it worth spending time improving their integration in the extension, with for example:

- functions to gather input from the user: e.g. reading code from the goal the user's cursor is on, or querying the editor buffer for a whole line of code;
- functions to handle output: e.g. displaying information in the display buffer or writing to the editor.

4.2.3 Evaluating the API

We can evaluate this API by implementing a number of small new features that do not exist in any other Agda extension. The hypothesis is that implementing these features with the API as described above, should be relatively simple, which we can evaluate by counting the lines as a crude but generally justified metric [12]. Examples of such features are:

- *Add Clause action*: Korkut and Christiansen implemented an "Add Clause" action, which, given a signature of a function, provides an initial skeleton for the definition of that function.

```
append : List A → List A → List A   -- Given signature                    Agda
append a b = {! !}                  -- Definition provided by the extension
```

- *Refactorings for dependently-typed programs*: We can also implement refactorings inspired from Barwell et al. [13]. They introduce refactorings such as adding an index to a data type definition, or relating two constructor parameters using a dependent type.
- *Hoogle-like lemma search*: this feature requests all names available in the current scope of the cursor and tries to unify them with a given type (or the requested type of the hole). This features needs to communicate a list of types back to the extension, which is then displayed in the display buffer. In contrast to refactorings, this features needs to communicate data structures that are not just a single term, which requires custom instances for `Editorable`.

- *Caching for proofs by reflection*: proof by reflection is a technique where functions mechanically generate proofs for a theorem [14]. Computing these proofs for large proof trees can be expensive, especially considering that these proofs may not change often. Therefore, it might be more efficient to inline the proof tree in the source to avoid doing the proof by reflection at elaboration-time. Inlining the proof tree could be implemented as an editor action in our extension.

4.3 Unifying architecture for all Agda extensions

Looking at the implementations of existing Agda extensions, a lot of the infrastructure for handling user commands, and responses from the Agda process are quite similar (up to differences in programming languages). This "core" part of every extension can be extracted and generalised such that the editor interactions are abstract. With a working Visual Studio Code extension, we could try adding support for another IDE – for example Atom[10], whose packages also need to be written in Javascript – to create an abstraction layer between the editor-specific implementations and the editor-agnostic Agda interfacing code. Additionally, if the "core" logic is designed in such a way that it can be compiled to both Haskell and Javascript, we could also add support for Neovim using the same library as Cornelis uses.

4.4 Improving the foreign function interface

During the development of the extension prototype thusfar, one major point of setback has been using the FFI. Currently, the FFI only provides one language pragma: `COMPILE JS`. This pragma takes a name of a function and the Javascript implementation for that function, and pastes it without semantic modification into the compiled output. The compiler has a flag `--js-verify` that checks for syntax errors in the compiled output, but this gives very little correctness guarantees. This flag cannot, for instance, verify whether FFI functions have the correct number of parameters or the correct type.

We propose to improve the FFI by making it directed by Typescript functions. We can take a Typescript file and generate an Agda type for each exported function of the module we import using the FFI tool. For each function, we can request the parameter types and return type from the Typescript compiler, then create a curried function call and curried function type, and insert those terms into an Agda file. When going into this direction, we can adopt Typescript types incrementally:

1. We can select a subset of Typescript's type system, for example akin to the types in System F limited to builtin types, that we want to convert to Agda. We would need to create an interface with the rather under-documented Typescript compiler API[11] to analyse source files and request the types of functions within.
2. Once the basis for the tool works, we can try expanding the tool's capabilities by generating ML-style modules for classes, and as such also try supporting types other than builtin types and type variables. This encoding for objects has been abundantly used in the extension already. Listing 12 shows roughly how a simple class in Typescript can be converted to a module in Agda. The constructor could be compiled to a function called `new`, every method to a function with the same name, and every type parameter to a declaration in a variable block.

[10] https://flight-manual.atom-editor.cc/hacking-atom/

[11] https://github.com/Microsoft/TypeScript/wiki/Using-the-Compiler-API

```typescript
class List<T> {
  constructor(length: number) {}
  public get(index: number): T {}
}
```

TypeScript

```
module List where
  private variable T : Set
  postulate
    t : Set → Set
    new : Nat → t T
    get : Nat → t T → T
```

Agda

Listing 12: Proposed translation of class definitions in Typescript to modules in Agda.

Typescript's type system has some features that Agda's type system does not. For example, Typescript has unlabelled sum types: `T | U`. With Agda's sum types (`T ⊎ U`), the left and right injection can still be discriminated, whereas in Typescript these types might not be discriminable. Another example is Typescript's intersection types: `T & U`, which creates a new object type that contain all fields that are common in `T` and `U`. Agda does not have row types, so finding the intersection of records is quite difficult. This leads us to believe that, even though research into this idea might be valuable, it is out of scope for this thesis.

4.5 Implementing/extending the LSP protocol for Agda

As mentioned earlier, existing Agda extensions all have custom logic for interfacing with the editors. The responses Agda provides are also custom. This means that if we want to support a new editor, then we need to write a lot of interfacing logic for that editor again. As mentioned in Section 2.3, the LSP protocol aims to solve this problem. There already exist implementations for an Agda language server with varying degrees of completeness:

- As mentioned in Section 2.1, the Agda Language Server (ALS)[12] is a project by the author of the original Visual Studio Code extension for Agda. It is integrated with the extension, and therefore it only has functionality that is not already implemented in the extension. The current version only includes hover functionality for displaying types of terms.
- For his Master's thesis, Stuijt Giacaman implemented agda-lsp – a language server designed that aims to provide provide real-time feedback to the users [15]. Agda-lsp achieves this through the use of a custom scope-checker for Agda, and avoiding the use of the type-checker, which can be slow for bigger source files. The scope-checker provides sufficient information to provide basic LSP functionality such as semantic highlighting and go-to-definition, but also functionality currently not found in any Agda extensions making use of the interaction mode: finding references of a given symbol, and diagnostics for unused symbols.

Even though there is plenty of future work that these projects suggest, it would be difficult to relate that work to the our non-LSP implementation for an Agda extension. We therefore deem this research direction out of scope for this thesis.

[12] https://github.com/agda/agda-language-server

5 Bibliography

[1] M. Dodds, 'What Works (and Doesn't) Selling Formal Methods'. Accessed: Mar. 13, 2026. [Online]. Available: https://www.galois.com/articles/what-works-and-doesnt-selling-formal-methods

[2] S. Juhošová, A. Zaidman, and J. Cockx, 'Pinpointing the learning obstacles of an interactive theorem prover', in *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*, 2025, pp. 159–170.

[3] R. Backx, 'A Grounded Theory Study of "How Interactive Theorem Prover Programmers Write Code"', Master's thesis, 2026.

[4] Stack Overflow, 'Stack Overflow Developer Survey 2025'. Accessed: Feb. 26, 2025. [Online]. Available: https://survey.stackoverflow.co/2025/

[5] Microsoft, 'language-server-protocol'. Accessed: Oct. 05, 2022. [Online]. Available: https://microsoft.github.io/language-server-protocol/

[6] G. Lomurno and idris2-lsp contributors, 'idris2: Language Server for Idris2'. Accessed: Dec. 28, 2025. [Online]. Available: https://github.com/idris-community/idris2-lsp

[7] L. de Moura and S. Ullrich, 'The Lean 4 Theorem Prover and Programming Language', in *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*, Berlin, Heidelberg: Springer-Verlag, 2021, pp. 625–635. doi: 10.1007/978-3-030-79876-5_37.

[8] E. Tassi, R. Tetley, and vsrocq contributors, 'vsrocq: Visual Studio Code extension for Coq'. Accessed: Nov. 27, 2025. [Online]. Available: https://github.com/rocq-prover/vsrocq

[9] E. J. Gallego Arias and rocq-lsp contributors, 'Rocq-lsp: Visual Studio Code Extension and Language Server Protocol for Rocq'. Accessed: Nov. 30, 2025. [Online]. Available: https://github.com/ejgallego/rocq-lsp

[10] J. K. Rask, F. P. Madsen, N. Battle, H. D. Macedo, and P. G. Larsen, 'The Specification Language Server Protocol: A Proposal for Standardised LSP Extensions', in *Proceedings of the 6th Workshop on Formal Integrated Development Environment, F-IDE@NFM 2021, Held online, 24-25th May 2021*, J. Proença and A. Paskevich, Eds, in EPTCS, vol. 338. 2021, pp. 3–18. doi: 10.4204/EPTCS.338.3.

[11] J. Korkut and D. T. Christiansen, 'Extensible type-directed editing', in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Type-Driven Development*, in TyDe 2018. St. Louis, MO, USA: Association for Computing Machinery, 2018, pp. 38–50. doi: 10.1145/3240719.3241791.

[12] J. Rosenberg, 'Some Misconceptions About Lines of Code', in *Proceedings of the 4th International Symposium on Software Metrics*, in METRICS '97. USA: IEEE Computer Society, 1997, p. 137.

[13] A. D. Barwell, C. Brown, M. S. Chang, C. Theocharis, and S. Thompson, 'Structural Refactorings for Exploring Dependently Typed Programming', in *Trends in Functional Programming: 25th International Symposium, TFP 2024, South Orange, NJ, USA, January 10–12, 2024, Revised Selected Papers*, South Orange, NJ, USA: Springer-Verlag, 2024, pp. 1–21. doi: 10.1007/978-3-031-74558-4_1.

[14] P. D. van der Walt, 'Reflection in agda', Master's thesis, 2012.

[15] W. L. Stuijt Giacaman, 'Can The Language Server Protocol Handle Dependent Types?', Master's thesis, 2024.